

Code-Layout, Readability and Reusability

Date	02 July 2026
Team ID	
Project Name	Darshan Ease – Temple Darshan Ticket Booking System
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the Darshan Ease codebase in terms of its structure, readability, and reusability. The backend follows the MVC pattern with clearly separated routes, controllers, models, and middleware, while the frontend uses reusable React components and a shared API service.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent 2-space indentation across both React and Express files.
2	Proper File Structure	Files and folders are logically organized	Yes	Backend split into routes/, controllers/, models/, middleware/, config/; frontend split into components/, services/.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names like name, email, password, product, order are self-explanatory.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. createOrder, getOrders, fetchProducts, handleRegister clearly describe their action.
5	Code Comments	Inline and block comments explain logic	Partial	Core logic works correctly but has very few inline comments explaining the "why" behind it.
6	Modular Design	Code is split into reusable functions/modules	Yes	Auth middleware, API service, and CRUD controllers are reused across multiple routes/pages.
7	No Redundant Code	Duplicate or unused code is removed	Partial	Minor duplicate code was removed during development. The current codebase is clean and organized.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	try/catch blocks in controllers and a JWT auth middleware return proper error responses (e.g. 401 Unauthorized).

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	authMiddleware.js	Node.js / Express, JWT	Protects orders, profile, and admin routes	High
2	API service (Axios instance)	React / JavaScript	Register, Login, Products, and Orders components	High
3	Mongoose schema pattern	MongoDB / Mongoose	User, Product, and Order models follow the same schema/CRUD pattern	High
4	Order/Product CRUD controller pattern	Node.js / Express	Reused controller structure for products and orders	Medium
5	Product/Booking list rendering component	React	Reused across the Products page and Products page and Orders page.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Well-organized MVC structure with clearly separated routes, controllers, models, and middleware.
Readability	4	Variable and function names are clear and consistent, but inline comments are minimal.
Reusability	4	Auth middleware, API service, and schema/controller patterns are reused across multiple modules.
Documentation / Comments	3	Code currently has very few comments explaining the logic; needs improvement before final submission.
Overall Score	4.0	Solid, functional codebase with a clean structure; main improvement areas are comments and removing minor redundant code.